JPEG Image Steganography for DRM

This is the basic documentation for the JPEG Image Steganography module developed by Alex Fotios as a demo of a JPEG DRM (Digital Rights Management) system based on steganography.

Why steganography

The main problem an image DRM system has to solve is how to resiliently encode identifying information in the image medium while keeping the medium as perceptually untouched as possible.

By definition, watermarks work by altering the image itself. There's various ways to make watermarks less perceptible, however, in general, watermarks that are not very perceptible are not that hard to defeat with some clone-brushing using tools like Photoshop and GIMP.

Steganography is a subtler yet more powerful way of encoding information in images. Steganography has to be by definition hidden to the naked eye and ideally be decodable only by using some fairly sophisticated tool or technique.

How it works

In the case of JPEG images the binary information to be steganographically encoded in the medium is multiplexed into the actual Y Cb Cr fabric of the JPEG by chroma-shifting pixels by a certain value (usually 1 or 2). The theory is that the human eye is more sensitive to luminance changes than to chrominance changes. Therefore changing the numeric value of Cb and/or Cr by a value of 1 or 2 while keeping the value of Y unchanged should be virtually imperceptible while giving us a way to encode arbitrary information in the image.

The steganography module that has been developed does both watermarking and pure steganography. Both DRM subsystems are configurable via a number of options.

There is an encoder subsystem that encodes the desired information into the image and a decoder subsystem that is used to decode the steganographic data out of an encoded image. Watermarked data is visually inspected. In case the watermark uses a very small font size, a zooming image viewer can be used.

Watermark data can be an arbitrarily long character string. The current implementation of the steganography subsystem supports a maximum string length of 8 ASCII (7-bit) chars.

Watermarking works by increasing the Y component of the involved pixels while leaving the Cb and Cr components unchanged.

Technologies used

The steganography module is developed in C, for the Linux platform, and uses SWIG to compile into a PHP loadable module. It can therefore be accessed by PHP scripts while providing the speed of a natively compiled program.

Pure steganography

Here's how steganography is actually implemented.

What we see here is a steganographically encoded JPEG image where Cb and Cr values, of the pixels used for encoding information, have been boosted by a value of 50, instead of the usual 1 or 2, in order to become visible.

As can be easily deduced, the information is encoded using a rudimentary bar code scheme. Each bar encodes two bits of information – one in its top half and one its bottom half. 1-bits are encoded as chroma shift-ups (chroma value + 50) and 0 bits are encoded as shift-downs (chroma value - 50).

There are 29 bars in total but only 28 are used for actually encoding information; the very first bar is called the “anchor bar” – more on its function below. Since each bar encoded 2 bits we have a total of

56 bits encoded in the image. If we use 7-bit ASCII chars, 56 bits encode exactly 8 chars; 4 in the upper half of the image and 4 in the lower half it.

As you can see the left and right extremities of the image do not contain any encoded info. The width of these extremities is configurable (see the "anti-crop image height/width percent" parameter) and is there in order to protect the image from cropping attacks.

The bit-encoding bars and the spaces between them need to be wide enough in order to counter image information deterioration due to scaling and recompressing – either because of attacks or due to normal/expected processing during the image instance's lifetime.

The very first bar is called the "anchor bar" and its function is to mark the beginning of the bit-encoding bars part of the image. It has a +1 chroma shift value compared to the regular bit-encoding bars and uses an alternating chroma-shift pattern in order to compensate for the stronger shift value (alternating shift patterns are less perceptible to the eye). The stronger shift value is needed in order to optimize the chances that the location of the anchor bar is accurately detected by the decoder. The anchor bar does not encode any information itself.

When an image is higher than it is wide the bars are encoded horizontally in order to utilize the JPEG's pixel real-estate more efficiently and therefore increase the resilience of the encoded information.

How to use it

The steganography module exposes two functions to the PHP runtime via SWIG. Any PHP script that imports the steganography2.php file can use those functions. These functions are:

- *steg_easy_apply2()*
- *steg_easy_decode2()*

Both of these functions receive a number of functions.

Encoder Options

Watermarking options

These are the parameters of *steg_easy_apply()*, with value ranges provided in parentheses:

- **Watermark:** This is the string that will be overlaid as a watermark in the image. It can be of any length although very long strings have not been tested (and they don't make sense for watermarking anyway)
- **Watermark Font size:** determines how big the letters of the watermark will be.
- **Watermark X step:** the horizontal distance (in pixels) between separate instances of the watermark string.
- **Watermark Y step:** the vertical distance (in pixels) between separate instances of the watermark string.

- **Old Watermarking:** (0, 1) When set to 1 it enables the old watermarking functionality, as implemented in the previous watermarking only steganography module. The parameter is provided for test and comparison purposes. If enabled (i.e. set to 1), the following two options are ignored.
- **Watermark Y-bump value:** How much the value of the Y component of the watermarked pixels goes up or down. The value can be positive or negative (positive = lighter, negative = darker)
- **Reactive Y-bump:** This can be either ON or OFF (1 or 0 respectively) and when ON the logic reacts intelligently to the pixel's current Y value. Specifically, it takes the absolute Y-bump value given above and makes it negative if the current Y value is high and positive when low.

Steganography options

- **Chroma Shift Steganography:** If set to ON (i.e. set to 1) the module steganographically encodes the "steganographic message" string, as given in the appropriate text field below, into the uploaded image.
- **Anti-crop Protection:** If set to OFF, image is encoded with no anti-crop anchor-bar and the "anti-crop percent" field, right below, is ignored.
- **Anti-crop image width/height percent:** the percentage of width (or height if height > width) of the image, on each side of it, that will not be used for encoding information in order to protect that information in the event the image is cropped. The bigger this value is the more protected the image is against more aggressive cropping attacks but the more fragile the information becomes to resizing and recompressing events/attacks.
- **Steganographic message:** This text-field receives the message string that is to be steganographically encoded. It can be up to 8 chars long.
- **Cb channel chroma shift:** Determines how much the Cb channel's chroma value goes up or down (up or down depends on the value of the bit being encoded)
- **Cr channel chroma shift:** Determines how much the Cr channel's chroma value goes up or down (up or down depends on the value of the bit being encoded)

Decoder options

- **Anti-Crop Decoding:** If set to OFF, earlier-style, no anchor-bar, no anti-crop gutter, decoding is performed. Encoding an image with no protection against cropping attacks results to steganography that is more resilient to resizing and recompression attacks as the encoded info is spread out over a larger area. If you think that your images are not likely to be cropped by your attackers, this option lets you take advantage of that fact. However, it is important to understand that the slightest cropping of the image will result in an image that the decoder won't be able to deal with. To successfully use this option the corresponding encoder option (Anti-crop Protection set to OFF/0) should be used when encoding the image in question.
- **Anti-Crop No-Crop Hint Decoding:** If set to ON, decoder assumes a given image, encoded with encoder's "anti-crop" option set to ON, has not been cropped and bases bar step guessing on image dimensions instead of the anchor-bar. This will improve decoding results if indeed the image has not been cropped at all. If set to OFF, the "Anti-crop percent" value (see below) is ignored.
- **Anti-crop image width/height percent:** the percentage of width (or height if height > width) of the image, on each side of it, that has not been used for encoding information in order to protect that

information in the event the image is cropped. This percentage should be EXACTLY the same as the one specified when the image was encoded.

- **Ignore Y:** This is an option that helps the decoder deal with high brightness/contrast of certain images or with brightness attacks. If set to ON, decoder does not perform Y component comparisons while looking for chroma shifted pixels and the "Y bound" option right below is ignored.
- **Y bound:** While the “Ignore Y” option is set to OFF, varying the Y bound value (e.g. 1, 5, 10, etc.) can also help deal with too bright or brightened images. Setting the bound value to 0 (zero) can also help sometimes. Tinkering with this value can also help decode noisy highly-scaled or low-quality images (e.g. JPEGs that have been compressed to a low quality setting).
- **Bounds based chroma spikes check:** If set to OFF, following three options are ignored. When set to ON, it can help correctly decode "noisy" images. "Bound" value can be further boosted to higher or lower values than the default (3).
- **Cb channel chroma shift:** should be EXACTLY the same value as the one used while encoding the image (see encoder options documentation above for explanation of what this option means).
- **Cr channel chroma shift:** should be EXACTLY the same value as the one used while encoding the image (see encoder options documentation above for explanation of what this option means).
- **Chroma shift check bound:** When this value is bigger than zero(0) it helps detect chroma shifted pixels whose value has gone a bit up or down due to scaling and/or recompression JPEG deterioration. However, false positives also increase as the value gets bigger. Optimal value depends on the specifics of the image being decoded.

Proposed testing protocol

- Take 10-20 representative images that have been encoded at a chroma shift value of 2 (use the same value on both the Cb and Cr channels).
- Modify all of the selected images in the same ways, always using the same specific settings, e.g. using the arbitrary numbers below:
    - simple resave at 70%, baseline optimized
    - brightness change to 15% and resave at 70%, baseline optimized
    - scale down by 10% and resave at 70%, baseline optimized
    - crop a small part and resave at 70%
    - scale down by 10%, crop a small part and resave at 70%
- Then decode each one of the test images and each one of the modifications of each image.
- Make sure you use the non-crop hint for un-cropped images as it decodes them better (this option should be pre-selected, i.e. be ON by default).
- Note and record image modifications that do not work well.
- For any problematic cases try and see if you can get the image to decode by changing the Y bound value to 0 or 10 (from the default 5) and also try to turn on the specific Y bounds checking option and modify the bound value.

Maintenance

The decoder dumps decoded images in the steg directory under the admin directory. You should delete all files in that directory periodically, either manually or via a cron job.

Development

All source code files contain inline comments and discussion.

Source code structure

The module's source code comes packaged as a gzipped tar. Under the root directory there is a directory named "src" which contains all source files. Under the "src" directory there is a directory named "jpeg-v4" that contains the source files of the original JPEG library used. Some of the original library's files have been heavily modified.

Important source files

The two most important files that contain most of the module's watermarking and steganographic functionality are:

- jccolor.c (under the "jpeg-v4" directory)
- jpeg.c (under the "src" directory)

Installation

Issue the typical "configure", "make", "make install", command sequence while on the root directory and then make sure your php.ini file includes a module reference for "steganography2".